Divide and Conquer

Divide and Conquer

The **divide-and-conquer** paradigm,

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

Base case: If the subproblems are small enough, just solve them by brute force.

Combine the subproblem solutions to form a solution to the original problem.

Questions

Use divide-and-conquer approach to implement

- binary search-tree algorithm.
- binary search algorithm.

Questions

Use divide-and-conquer approach to implement

- binary search-tree algorithm.
- binary search algorithm.

	Un-Balanced	Balanced
Binary search-tree		
Binary search		

Questions

Use divide-and-conquer approach to implement

- binary search-tree algorithm.
- binary search algorithm.

	Un-Balanced	Balanced
Binary search-tree	**T(n)=T(n-1)+1**	
Binary search		

Questions

Use divide-and-conquer approach to implement

- binary search-tree algorithm.
- binary search algorithm.

	Un-Balanced	Balanced
Binary search-tree	T(n)=T(n-1)+1	**T(n)=T(n/2)+1**
Binary search		

Questions

Use divide-and-conquer approach to implement

- binary search-tree algorithm.
- binary search algorithm.

	Un-Balanced	Balanced
Binary search-tree	T(n)=T(n-1)+1	T(n)=T(n/2)+1
Binary search	**T(n)=T(n/2)+1**	

Questions

Use divide-and-conquer approach to implement

- binary search-tree algorithm.
- binary search algorithm.

	Un-Balanced	Balanced
Binary search-tree	T(n)=T(n-1)+1	T(n)=T(n/2)+1
Binary search	**T(n)=T(n/2)+1**	

Best case?

Questions

Use divide-and-conquer approach to implement

- binary search-tree algorithm.
- binary search algorithm.

	Un-Balanced	Balanced	Best Case
Binary search-tree	T(n)=T(n-1)+1	T(n)=T(n/2)+1	**T(n)=O(1)**
Binary search	T(n)=T(n/2)+1		**T(n)=O(1)**

ALGORITHMIC RECURRENCES

A recurrence T(n) is ***algorithmic*** if for every sufficiently large ***threshold*** constant $n_o \geq 0$:

- For all $n < n_0$,

- For all $n \geq n_0$,

ALGORITHMIC RECURRENCES

A recurrence T(n) is ***algorithmic*** if for every sufficiently large ***threshold*** constant $n_o \geq 0$:

- For all $n < n_0$, $T(n) = \Theta(1)$.
    - We consider the running time as constant for small problem sizes.

- For all $n \geq n_0$,

ALGORITHMIC RECURRENCES

A recurrence T(n) is ***algorithmic*** if for every sufficiently large ***threshold*** constant $n_o \geq 0$:

- For all $n < n_0$, $T(n) = \Theta(1)$.
    - We consider the running time as constant for small problem sizes.
- For all $n \geq n_0$, every path of recursion terminates in a defined base case within a finite number of recursive invocations. [The recursive algorithm terminates.]

METHODS FOR SOLVING RECURRENCES

- **Substitution method:**
- **Recursion-tree method:**
- **Master method:**
- **Akra-Bazzi method:**

SUBSTITUTION METHOD

1. Guess the solution.
2. Use induction to find the constants and show that the solution works.

SUBSTITUTION METHOD

1. Guess the solution.
2. Use induction to find the constants and show that the solution works.

Example:

Determine an asymptotic upper bound on $T(n) = 2T(\lfloor n/2 \rfloor) + n$

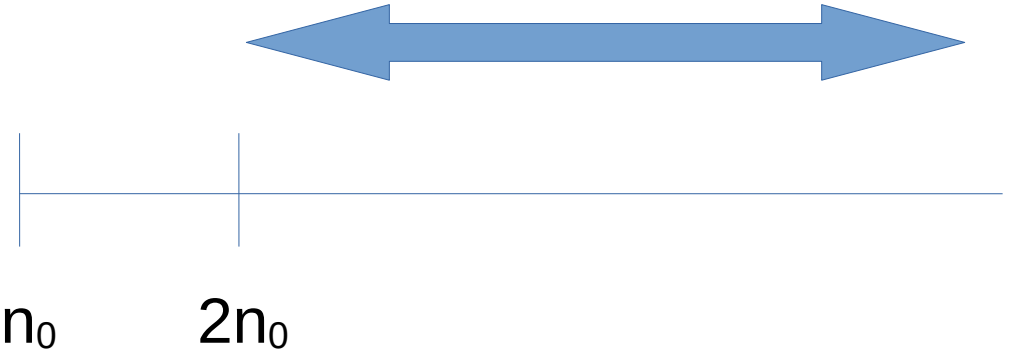
Floor function ensures that T is defined over integers.

Guess: T(n)=O(n lg n).

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

We need to show that $T(n) \leq cn \lg n$, for some constant $c>0$ for all $n_0 \leq n$

$$\begin{aligned} T(n) &\leq 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n - cn + n \\ &\leq cn \lg n \ , \end{aligned}$$

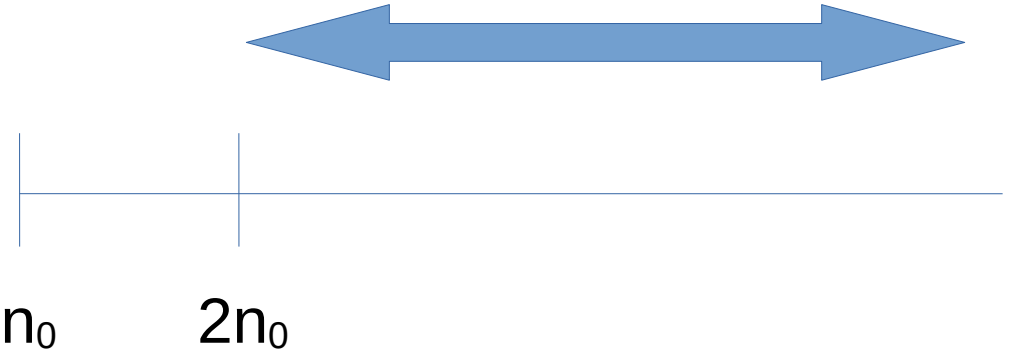


What about the base cases?

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

We need to show that $T(n) \leq cn \lg n$, for some constant $c>0$ for all $n_0 \leq n$

$$\begin{aligned} T(n) &\leq 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n - cn + n \\ &\leq cn \lg n \ , \end{aligned}$$



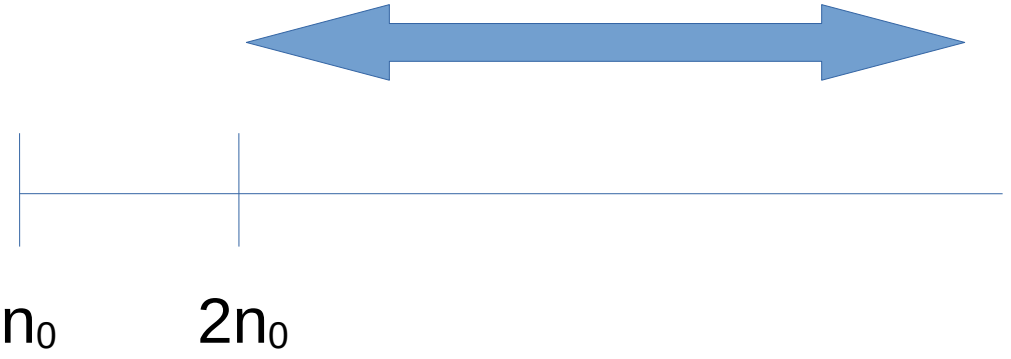
Base case: $n_0 \leq n < 2n_0$

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

We need to show that $T(n) \leq cn \lg n$, for some constant $c>0$ for all $n_0 \leq n$

Take the value of c such that the base case go through.

Can $n_0=1$? NO



Base case: $n_0 \leq n < 2n_0$

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

We need to show that $T(n) \leq cn \lg n$, for some constant $c>0$ for all $n_0 \leq n$

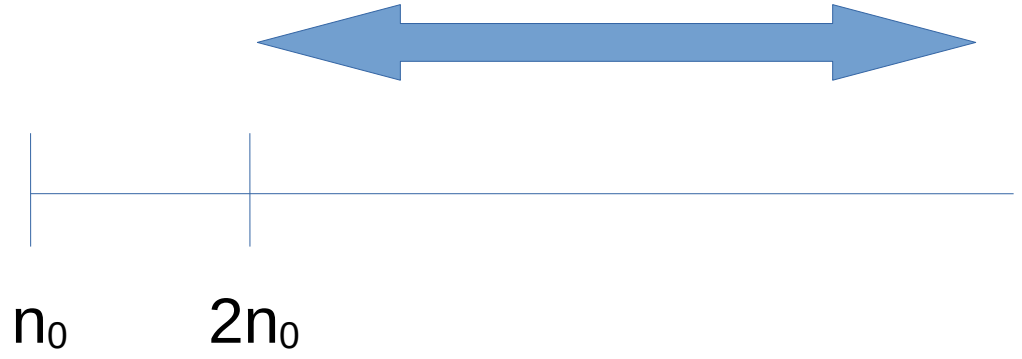
Take the value of c such that the base case go through.

Can $n_0=1$? NO

Set $n_0=2$, therefore n=2 and n=3 form the base cases.

Set c = max {T(2), T(3)}

(from the recurrence)



Base case: $n_0 \leq n < 2n_0$

SUBSTITUTION METHOD (continued)

$$
\begin{aligned}
T(n) &\le 2(c\lfloor n/2\rfloor \lg(\lfloor n/2\rfloor)) + n \\
&\le cn\lg(n/2) + n \\
&= cn\lg n - cn\lg 2 + n \\
&= cn\lg n - cn + n \\
&\le cn\lg n\ ,
\end{aligned}
$$

Is $O(n^2)$ a good guess?
What about $O(n)$?

MAKING A GOOD GUESS

No general way to make a good guess. Experience helps. Can also draw out a recursion tree.

Question.

Determine an asymptotic upper bound on $T(n) = 2T(\lfloor n/2 \rfloor) + n$

Floor function ensures that T is defined over integers.

Guess: T(n)=O(n lg n).

What if you replace f(n) from n to Θ(n)?

When the additive term uses asymptotic notation.

- Name the constant in the additive term.
- Show the upper (O) and lower (Ω) bounds separately. Might need to use different constants for each.

When the additive term uses asymptotic notation.

- Name the constant in the additive term.
- Show the upper (O) and lower (Ω) bounds separately. Might need to use different constants for each.

Example:

$T(n) = 2T(n/2) + \Theta(n)$. If we want to show an upper bound of $T(n) = 2T(n/2) + O(n)$, we write $T(n) \leq 2T(n/2) + cn$ for some positive constant c.

Important: We get to name the constant hidden in the asymptotic notation (c in this case), but we do ***not*** get to choose it, other than assume that it's enough to handle the base case of the recursion.

SUBSTITUTION METHOD (CONTINUED)

Upper bound:

Guess: for some positive constant d. This is the inductive hypothesis.

Important: We get to both name and choose the constant in the inductive hypothesis (d in this case). It OK for the constant in the inductive hypothesis (d) to depend on the constant hidden in the asymptotic notation (c).

Substitution:

$$
\begin{aligned}
T(n) &\le 2T(n/2) + cn \\
&= 2\left(d\frac{n}{2}\lg\frac{n}{2}\right) + cn \\
&= dn\lg\frac{n}{2} + cn \\
&= dn\lg n - dn + cn \\
&\le dn\lg n \qquad \text{if } -dn + cn \le 0, \\
&\qquad\qquad\qquad\qquad d \ge c
\end{aligned}
$$

Therefore, $T(n) = O(n\lg n)$.

SUBSTITUTION METHOD (CONTINUED)

Lower bound:

Substitution:

$$
\begin{aligned}
T(n) &\geq 2T(n/2) + cn \\
&= 2\left(d\frac{n}{2}\lg\frac{n}{2}\right) + cn \\
&= dn\lg\frac{n}{2} + cn \\
&= dn\lg n - dn + cn \\
&\geq dn\lg n \qquad \text{if } -dn + cn \geq 0\,, \\
&\qquad\qquad\qquad\qquad d \leq c
\end{aligned}
$$

Therefore, $T(n) = \Omega(n \lg n)$.

[For this particular recurrence, we can use c for both the upper-bound and lower-bound proofs. That won't always be the case.]

SUBTRACTING A LOW-ORDER TERM

Might **guess the right asymptotic bound**, but the **math doesn't go** through in the proof.

Example:

$$T(n) = 2T(n/2) + \Theta(1).$$

SUBTRACTING A LOW-ORDER TERM

Might **guess the right asymptotic bound**, but the **math doesn't go** through in the proof.

Example:

$T(n) = 2T(n/2) + \Theta(1)$. Guess that $T(n) = O(n)$

SUBTRACTING A LOW-ORDER TERM

Might guess the right asymptotic bound, but the math doesn't go through in the proof.

Example:

$T(n) = 2T(n/2) + \Theta(1)$. Guess that $T(n) = O(n)$

$$
\begin{aligned}
T(n) &\le 2(c(n/2)) + \Theta(1) \\
&= cn + \Theta(1) .
\end{aligned}
$$

But this doesn't say that $T(n) \le cn$ for *any* choice of c.

What do we do?

SUBTRACTING A LOW-ORDER TERM

Might guess the right asymptotic bound, but the math doesn't go through in the proof.

Example:

$T(n) = 2T(n/2) + \Theta(1)$. Guess that $T(n) = O(n)$

$$\begin{aligned} T(n) &\le 2(c(n/2)) + \Theta(1) \\ &= cn + \Theta(1) . \end{aligned}$$

But this doesn't say that $T(n) \le cn$ for *any* choice of c.

Try a larger guess?

What do we do?

SUBTRACTING A LOW-ORDER TERM

Might guess the right asymptotic bound, but the math doesn't go through in the proof. **Resolve by subtracting a lower-order term.**

Example:

$T(n) = 2T(n/2) + \Theta(1)$. Guess that $T(n) = O(n)$

$$\begin{aligned} T(n) &\le 2(c(n/2)) + \Theta(1) \\ &= cn + \Theta(1) . \end{aligned}$$

But this doesn't say that $T(n) \le cn$ for *any* choice of c.

SUBTRACTING A LOW-ORDER TERM
(continued)

Could try a larger guess, such as $T(n) = O(n^2)$, but not necessary. We're off only by $\Theta(1)$, a lower-order term. Try subtracting a lower-order term in the guess: $T(n) \leq cn - d$, where $d \geq 0$ is a constant:

$$\begin{aligned} T(n) &\leq 2(c(n/2) - d) + \Theta(1) \\ &= cn - 2d + \Theta(1) \\ &\leq cn - d - (d - \Theta(1)) \\ &\leq cn - d \end{aligned}$$

as long as d is larger than the constant in $\Theta(1)$.

SUBTRACTING A LOW-ORDER TERM
(continued)

Why subtract off a lower-order term, rather than add it? Notice that it's subtracted twice. Adding a lower-order term twice would take us further away from the inductive hypothesis. Subtracting it twice gives us , and it's easy to choose constants to make that inequality hold.

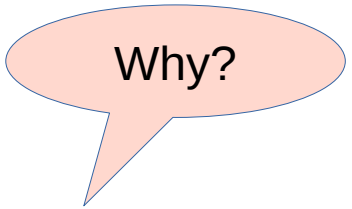
Important: Once again, we get to name and choose the constant in the inductive hypothesis. And we also get to name and choose the constant that we subtract off.

SUBTRACTING A LOW-ORDER TERM
(continued)

Be careful when using asymptotic notation

A false proof for the recurrence $T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n)$, that $T(n) = O(n)$:

$$
\begin{aligned}
T(n) &\le 2 \cdot O(\lfloor n/2 \rfloor) + \Theta(n) \\
&= 2 \cdot O(n) + \Theta(n) \\
&= O(n) . \qquad \Longleftarrow \textit{wrong!}
\end{aligned}
$$



SUBTRACTING A LOW-ORDER TERM
(continued)

Be careful when using asymptotic notation

A false proof for the recurrence $T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n)$, that $T(n) = O(n)$:

$$
\begin{aligned}
T(n) &\le 2 \cdot O(\lfloor n/2 \rfloor) + \Theta(n) \\
&= 2 \cdot O(n) + \Theta(n) \\
&= O(n) . \qquad \Longleftarrow \textit{wrong!}
\end{aligned}
$$

This "proof" changes the constant in the Θ-notation. Can see this by using an explicit constant. Assume $T(n) \le cn$ for all $n \ge n_0$:

$$
\begin{aligned}
T(n) &\le 2(c \lfloor n/2 \rfloor) + \Theta(n) \\
&\le cn + \Theta(n) ,
\end{aligned}
$$

but $cn + \Theta(n) > cn$.

Recurrences, when not equations?

- The recurrence only states the upper bound on T(n). Therefore, we can express its solution using O-notation.

$$T(n) \leq 2T(n/2) + \Theta(n)$$

Recurrences, when not equations?

- The recurrence only states the upper bound on T(n). Therefore, we can express its solution using O-notation.

$$T(n) \leq 2T(n/2) + \Theta(n)$$

- The recurrence only states the lower bound on T(n). Therefore, we can express its solution using Ω-notation.

$$T(n) \geq 2T(n/2) + \Theta(n)$$